

GPU Accelerated Matrix Factorization for Recommender Systems

Doruk Kilitcioglu*, Nicholas Greenquist†, Mohamed Zahran‡, Anasse Bari§

Courant Institute of Mathematical Sciences

Department of Computer Science

New York University, USA

Email: *doruk.kilitcioglu@nyu.edu, †ntg251@nyu.edu, ‡mzahran@cs.nyu.edu, §abari@nyu.edu

Abstract— Matrix Factorization (MF) is a popular algorithm used to power many recommender systems. Efficient and scalable MF algorithms are essential in order to train on the massive datasets that large scale recommender systems utilize. Graphics Processing Unit (GPU) technology has become very popular in recent years and has become widely used in machine learning. The massive parallelism GPUs offer creates an opportunity to develop an accelerated MF algorithm. This paper presents cu2rec, a matrix factorization algorithm written in CUDA. cu2rec implements a parallel version of Stochastic Gradient Descent (SGD) to solve large scale MF problems. cu2rec utilizes multiple advanced techniques to harness better performance from a GPU. These include aggressive use of constant memory for hyper-parameters and registers for heavily reused values, a sparse matrix data structure, a reduction sum total loss kernel, a novel approach to parallel lock-free updating of feature weights with minimized global memory writes, and fairness across weight updates using user index striding. With a single NVIDIA GPU, cu2rec can be 10x faster than state of the art sequential algorithms while reaching similar error metrics.

Index Terms—recommender system, matrix factorization, GPU, CUDA, SGD

I. INTRODUCTION

Matrix Factorization decomposes a rating matrix R of shape $m \times n$ into two feature matrices, P and Q . P ($m \times f$) and Q ($n \times f$), where f is the number of factors, are the learned feature matrices[1]. The goal of MF is to find the best P and Q matrices such that $R \approx P \times Q^T$, as shown in Figure 1. In state of the art models, it also involves finding U ($m \times 1$ matrix of user biases) and B ($n \times 1$ matrix of item biases), with an additional global bias μ which is set to the mean of all ratings.

MF is incredibly popular and has been used for serving recommendations by companies such as Amazon and Netflix[2]. In the era of big data, the datasets that MF is being asked to tackle are getting orders of magnitude larger than even the Netflix Dataset[3]. In order to keep up, MF algorithms need to be fast, scalable, and able to handle massive amounts of data.

Today, GPUs are being used more and more to parallelize algorithms that need to run on big data. GPUs have much more compute power than CPUs and work well performing the same instruction on multiple pieces of data (SIMD)[4]. GPUs originally were used for heavy graphical work but have become a staple of large machine learning models[5]. Because

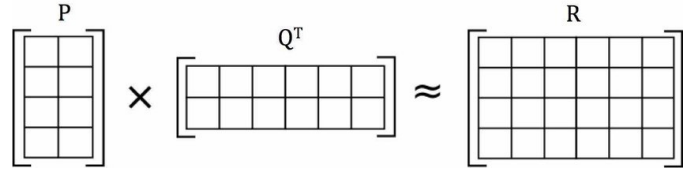


Fig. 1. Matrix Factorization

of the explosion of GPU usage across much of machine learning, we wanted to see if they could be useful in the sub-field of recommender systems.

There are many different algorithms to solve the MF problem for recommender systems. In this paper, we focus on Stochastic Gradient Descent (SGD). SGD has been shown to converge the fastest and is the most popular form of optimizer algorithm for recommender system specific MF, popularized by Koren [3]. SGD updates the values in P and Q by using the gradients of the loss function for each estimated rating in R compared to the real value. For recommender systems, it was shown that MF should update weights by computing the loss only on nonzero ratings[6]. As such, MF algorithms for recommender systems are memory bounded by the number of nonzero ratings in the matrix R , not the full size of the matrix ($m \times n$).

In this paper, we first examine the literature of existing approaches to parallel MF for recommender systems. Next, we show a novel parallel implementation of SGD (cu2rec) that takes advantage of many of the GPU's features and also utilizes a variety of advanced programming techniques. After discussing the implementation details, we describe the experimental setup where we test cu2rec against various sequential implementations of SGD MF on two different datasets. The results show that cu2rec is 20-40x faster than a naive sequential SGD implementation and 6-10x faster than one of the most popular recommender system library's MF implementation. We conclude the paper with a discussion of future work and other matrix factorization algorithms.

II. RELATED WORK

The Matrix Factorization approach to Recommender Systems has been used with a variety of different user-item

rating matrices, and Singular Value Decomposition (SVD) [7] is perhaps the most influential algorithm in the history of Recommender Systems. It was popularized by the Netflix Prize[3], and still remains used today. We use the sparse biased SVD used by Koren et al. [1].

The go-to dataset for Recommender Systems is the MovieLens[8] dataset, which has grown from 200k ratings to 20M ratings over the years. We use MovieLens and we support the previously mentioned Netflix Prize dataset[3], but it should also be noted that the Netflix Prize dataset has been removed from circulation due to privacy concerns, and will not be the focal point of our comparisons.

There are many sequential implementations of SGD, and there have been many attempts at parallelizing SGD using multi-core, distributed, and GPU accelerated systems. We have outlined the most outstanding work below.

A. Sequential

Surprise[9] is one of the most popular recommender system libraries for Python. It uses Cython, which allows algorithms to be written in C and be called from Python, resulting in C-levels of performance. It has numerous optimizations built in to accelerate the training of the biased SVD[1], which makes it one of the fastest sequential implementations. We compare our results to it in order to benchmark our implementation.

B. Multi-core

In multi-core solutions, we take particular note of Hogwild[10], which is a lock-free multi-core approach that works well with sparse environments such as matrix factorization in the context of recommender systems. Hogwild forms the basis of our approach.

The traditional method of parallelizing SGD requires locks, wherein the set of parameters that is used for one step of SGD is locked from being read until an update is finished. In the context of recommender systems, however, the sparsity of the matrix means that the set of parameters for a randomly chosen user-item pair is rarely overlapping with another randomly chosen pair. Recht et al. [10] exploits this idea to provide a lock-free, shared memory approach. Since multiple cores in a single GPU also share a single global memory, these ideas translate well for us.

C. Distributed

For distributed solutions, we look at NOMAD[11], which is a distributed implementation of the ideas outlined in Recht et al. [10]. It removes the need of a parameter server as outlined in the earlier work[12]. It only uses a number of asynchronous worker machines, each having access to a different portion of the data. Each worker machine owns one portion of the users, and that assignment never changes. Each worker machine also owns one portion of the items. Whenever the worker machine is done with one or more of its items, it sends the items to another worker machine.

This approach is lock-free because no two machines access the same users or the same items at the same time, and therefore updates can be made in parallel.

D. GPU

GPUs are a relatively new addition to matrix factorization for recommender systems. Of particular note are two solutions called CuMF-SGD[13] and CuMF-ALS[14]. Similar to our paper, the authors try to exploit GPUs for matrix factorization by using SGD and ALS respectively. We used these tools to compare our approach to theirs, which will be explained under evaluation.

We also tried Implicit[15], a MF library powered by CUDA, that works on implicit ratings only. Items are given 1 as a rating if they are consumed, or 0 if they aren't. Unfortunately, this discrepancy makes it impossible to compare results, as implicit ratings fundamentally uses different error metrics than explicit ratings.

E. SGD vs ALS

The jury is still out on whether or not SGD or Alternating Least Squares (ALS)[16] is more parallelizable. Tan et al. [14] claim that partitioning schemes such as those outlined in [11] do not scale with the number of cores a GPU works with, and that each iteration of SGD is inherently sequential and therefore it is better to use ALS.

In contrast, Recht et al. [10] and Yun et al. [11] argue that in the context of sparse matrix factorization in recommender systems, multiple SGD updates can be carried out simultaneously in parallel, and that leads to a higher potential for parallelism.

In our work, we utilize SGD as our workhorse, with modifications that allow us to use the ideas outlined in Recht et al. [10] to be used with GPUs.

III. IMPLEMENTATION

A. SGD for Recommender system Matrix Factorization

For the task of rating prediction, we use the biased SVD popularized by Koren et al. [1]. For each user u and item i pair, the estimated rating $\hat{R}_{ui}$ (which is the rating of item i by user u) and the error for that rating E_{ui} (which is the error of the rating of item i by user u) are defined by:

$$\hat{R}_{ui} = \mu + U_u + I_i + Q_i^T P_u \quad (1)$$

$$E_{ui} = R_{ui} - \hat{R}_{ui} \quad (2)$$

where U_u is the user bias (which is a single float), I_i is the item bias (also a single float), μ is the global mean, and P_u and Q_i are the user and item vectors in P and Q respectively.

Our total loss function is the mean squared error of ratings, plus the regularization terms for the 4 matrices, which are the squared Frobenius norms of the matrices:

$$L = \frac{1}{n} \sum_{i=1}^n (R_i - \hat{R}_i)^2 + \lambda_p \|P\|_2^2 + \lambda_q \|Q\|_2^2 + \lambda_u \|U\|_2^2 + \lambda_i \|I\|_2^2 \quad (3)$$

For SGD, we use the loss per item:

$$L_{ui} = E_{ui}^2 + \lambda_p \|P_u\|^2 + \lambda_q \|Q_i\|^2 + \lambda_u \|U_u\|^2 + \lambda_i \|I_i\|^2 \quad (4)$$

As with regular gradient descent, the update equation of any parameter x with learning rate η is defined by

$$x \leftarrow x - \eta \frac{\partial L}{\partial x} \quad (5)$$

In our case, the parameters are each element of the P , Q , U , and I matrices. Below are the respective partial derivatives that are used with the update equation above:

$$\frac{\partial L_{ui}}{\partial P_u} = -(E_{ui}Q_i - \lambda_p P_u) \quad (6)$$

$$\frac{\partial L_{ui}}{\partial Q_i} = -(E_{ui}P_u - \lambda_q Q_i) \quad (7)$$

$$\frac{\partial L_{ui}}{\partial U_u} = -(E_{ui} - \lambda_u U_u) \quad (8)$$

$$\frac{\partial L_{ui}}{\partial I_i} = -(E_{ui} - \lambda_i I_i) \quad (9)$$

B. Parallel SGD Challenges

As explained before in the related work, Tan et al. [14] claim that each iteration of SGD is inherently sequential, whereas Recht et al. [10] and Yun et al. [11] argue that in the context of sparse matrix factorization in recommender systems, multiple SGD updates can be carried out simultaneously in parallel.

In our implementation, we utilized ideas similar to those outlined in Recht et al. [10], but modified it to be used with GPUs. Because the matrices we are dealing with are very sparse, we were able to parallelize SGD updates across multiple users and items. We chose to parallelize over users, which in reasonable-sized datasets such as ML-20M, is much higher than the number of CUDA cores in modern GPUs. As a result, we can utilize our GPUs by choosing a random item per user to update.

We faced a few issues with this approach specific to GPUs, which warranted us to create our own flavor of the parallel SGD algorithm (see algorithm 1). The first issue we had was with multiple updates to the same item. We were running atomic operations for the updates to the item matrix, which meant that our code was slow, and worse, there were too many updates on the most popular items. This led to an overly drastic change for the item vectors, making it very hard to balance.

That is when we decided to get rid of the atomicity and have a race condition on the SGD updates, making it so that only one update per item would stick, and the previous update attempts would be void. We called this version of the algorithm Early Bird Loses the Gradient.

After analyzing our kernel, we noticed that the users with higher user ids had lower error than users with lower user ids. This was the result of how blocks are scheduled in CUDA: blocks with lower block ids (which have the lower user ids) get scheduled earlier than higher block ids. As a result, a lot of the item updates that were done by earlier users were being overwritten, and the algorithm was overfitting to the later users.

To combat this issue, we introduced user id striding with each iteration. Each thread handles user $tid + (stride * iters) \bmod N$, where tid is the thread id and N is the total

number of users. This results in fair item updates with respect to the users.

Another improvement came from not allowing multiple updates to the same item, because that results in more global memory accesses. For that, we added a binary array of all items, and whenever an item is updated, we set its updated value to true. Note that this is done with regular checks, not a costly *atomicCAS*, because it is not a problem if there is a race condition and it gets overwritten a couple of times. It still is empirically faster, and we call this novel version of the algorithm Early Bird Gets the Gradient (EBGTG).

Also worth mentioning is that we don't need to compute the total loss every iteration. It is an expensive operation, and we only do it every couple of iterations to modify the learning rate if the SGD has stopped learning. Each thread in the SGD kernel just calculates the error for the selected user-item pair, resulting in a heavy speedup compared to calculating the error on the whole training set.

C. Data Preparation

1) *Sparse Matrix Representation*: For even decently large datasets, the full ratings matrix R is too big to fit in memory. This is doubly true when we are dealing with GPU memory, as we cannot physically add more memory to a GPU.

In order to represent the ratings matrix R , we use the Compressed Sparse Row (CSR) format. CSR matrices are defined by three arrays: *indptr*, *indices*, and *data*. The item indices for user u are stored in *indices[indptr[u] : indptr[u + 1]]* and their corresponding ratings are stored in *data[indptr[u] : indptr[u + 1]]*. This allows for efficient indexing into the matrix given a user, which plays into how we construct our kernel.

When representing users with no ratings, we repeat the same value in the *indptr* array, and handle that as a special case in our kernel.

2) *Input Data*: Our program accepts CSV files that are formatted as *user_id, item_id, rating* tuples, wherein both *user_id* and *item_id* are sequential numerical ids starting from 1, and are sorted based on *user_id*. For convenience, we provide a script that convert non-sequential ids to sequential ids, and a script that sorts the tuples based on *user_id*.

The program also needs two different files, one as a training set with which the MF model is trained, and a test set with which the model is evaluated. We use a completely random split across all tuples for generating these files, and split the data into 80% training and 20% test set. We also provide a script to split the data.

D. cu2rec

The cu2rec code is organized into 3 main parts: reading in data into sparse matrix form and moving it to the GPU, using SGD and Loss kernels to train the MF model, and moving data back to the host in order to write the trained components to files.

When a user runs cu2rec, they need to supply the train and test CSV files. cu2rec starts by reading both files into a

vector of Rating structs that live in the host memory. It then converts each vector of Ratings into a CSR representation. Next, it moves these matrices into the GPU's memory. Once the memory allocation is complete, cu2rec then loads in the hyper-parameters from a config file into constant memory.

Next, cu2rec calls a *train* function that is responsible for using the sparse rating matrices to train a MF model. *train* starts by initializing all of the components randomly using a normal distribution with mean of 0 and standard deviation of 1. For the P and Q matrices, these values are normalized by the number of factors the model will use. Next, all of these components are moved to the GPU's global memory using standard CUDA API functions. Each is wrapped in a helper function to check for CUDA errors. In addition to moving needed memory to the GPU, cu2rec also initializes a CUDA Random object and other needed variables to handle an adaptive learning rate. Finally, the iterative training of the model can begin.

The main training loop in cu2rec does *totalIterations* loops over two main steps: update components using SGD and compute total losses. cu2rec parallelizes over the number of users in the matrix for both steps. At the beginning of each iteration, the SGD Kernel is called (algorithm 1). Next, the algorithm checks if it is time to test the updated model on the test ratings (algorithm 2). This is only done periodically, as computing the total losses is expensive because it needs to compute the error on all ratings, not just one rating per user such as in SGD. After computing the new error metrics, cu2rec checks to see if the learning rate should be lowered by checking a patience counter. Finally, cu2rec swaps the pointers of the updated Q with the old Q and updated *itemBias* with the old *itemBias* components because we want to use the new values for the next round of updating. We do not need to keep copies of P and *userBias* matrices as the updates are simply done on the original matrices. We only keep a target version of Q and *itemBias* as updates will have race conditions between threads. More is discussed about this in the next section, Early Bird Gets the Gradient.

Once *totalIterations* of the training loop are complete, it is time to save the trained model and free all necessary memory. First, all the trained components are copied back to the host variables. Next, the CUDA variables are freed along with all host variables that are not part of the trained model. Outside of the *train* method, the main code is responsible for writing to file all necessary components of the model that can be used to serve recommendations. The necessary components to write to file are: P , Q , *userBias*, *itemBias*, and *globalBias*. Once all have been written to a file, cu2rec's final steps are to free those variables memories on the host and terminate the program.

1) *Early Bird Gets the Gradient*: Due to the sequential nature of SGD, we had to come up with a technique to implement SGD with multiple threads potentially trying to update the features of the same item. In SGD, a single error is computed for a rating and both the user corresponding to that rating and the item have their feature matrices and bias

Algorithm 1 SGD

```

1: function SGDKERNEL
2:    $x = (blockDim.x * blockDim.x + threadIdx.x +$ 
       $startUser) \% nRows$ 
3:
4:    $y_i = \text{RANDOMITEM}(x)$ 
5:
6:    $y = \text{indices}[y_i]$ 
7:    $ub = \text{userBias}[x]$ 
8:    $ib = \text{itemBias}[y]$ 
9:
10:   $pred = \text{GETPREDICTION}(f, p, q, ub, ib, gb)$ 
11:   $error = \text{data}[y_i] - pred$ 
12:
13:   $earlyBird = !\text{itemIsUpdated}[y]$ 
14:   $\text{itemIsUpdated}[y] = \text{true}$ 
15:
16:  for  $i = 0$  to  $f$  do
17:     $pIdx = \text{INDEX}(x, i, f)$ 
18:     $qIdx = \text{INDEX}(y, i, f)$ 
19:     $pOld = P[pIdx]$ 
20:     $qOld = Q[qIdx]$ 
21:
22:     $P[pIdx] = pOld + \eta * (error * qOld - \lambda_p p_u * pOld)$ 
23:
24:    if  $earlyBird$  then
25:       $QTrgt[qIdx] = qOld + \eta * (error * pOld -$ 
         $\lambda_q q_i * qOld)$ 
26:    end if
27:  end for
28:
29:   $\text{userBias}[x] = \text{userBias}[x] + \eta * (error - \lambda_u U_u * ub)$ 
30:
31:  if  $earlyBird$  then
32:     $\text{itemBiasTrgt}[y] = ib + \eta * (error - \lambda_i I_i * ib)$ 
33:  end if
34: end function

```

Algorithm 2 Loss

```

1: function LOSSKERNEL
2:    $x = blockDim.x * blockDim.x + threadIdx.x$ 
3:
4:    $p = P[x * f]$ 
5:    $ub = \text{userBias}[x]$ 
6:
7:   for  $i = \text{indptr}[x]$  to  $\text{indptr}[x + 1]$  do
8:      $\text{itemId} = \text{indices}[i]$ 
9:      $pred = \text{GETPREDICTION}(f, p, q, ub, ib, gb)$ 
10:     $error[i] = \text{data}[i] - pred$ 
11:  end for
12: end function

```

weights updated. Because cu2rec parallelizes over users and each user picks a random item they have rated, the same item vector (especially popular items) is highly likely to be picked

Algorithm 3 Prediction

```
1: function GETPREDICTION( $f, p, q, ub, ib, gb$ )
2:    $pred = gb + ub + ib$ 
3:   for  $i = 0$  to  $f$  do
4:      $pred = pred + q[i] * p[i]$ 
5:   end for
6:   return  $pred$ 
7: end function
```

by multiple threads. The chance of this becomes near certain with non-trivial amounts of users and is guaranteed if you have more users than items due to the Pigeon Hole principle.

In order to handle race conditions on the same memory, we created the novel Early Bird Gets the Gradient (EBGTG) technique. In the SGD Kernel (algorithm 1), each thread picks a random item from the user's rated items. Then, it computes the error using the current feature matrices and takes the difference with the true rating. Next, it uses this error to update all of the features. Updating the P matrix and $userBias$ values will never have write conflicts since each thread is responsible for a single user. However, updates to values in Q and $itemBias$ matrices requires special care.

In EBGTG, the first thread to pick an item wins the race to write to that item. To implement this, we created a new boolean array, $itemIsUpdated$, that stores a true or false value for if an item's features have already been updated. When a thread selects a random item, they check the value in the array and set a local $earlyBird$ variable to *true* if they 'won the race.' They then set the value for this item to *true* so other threads will set their $earlyBird$ variables to *false*. Threads then only spend time doing f global memory writes to Q and one global memory write to $itemBias[y]$. Even if two threads arrive at the same time, see $itemIsUpdated[i]$ is false, and both update item i , the last value written is what the value will be with no error caused.

However, it should be noted that due to warp divergence, even if a single thread in a warp is selected as the early bird, all threads in the warp are blocked until the early bird thread is finished updating $QTrgt$. Warp divergence is a unwanted occurrence in GPUs when branch conditions are introduced into kernels. In a GPU, threads are bundled into a group of threads, usually 32, that is called a warp. Every thread in a warp executes each instruction of a kernel in lockstep. When a conditional is reached, the GPU has no choice but to block all threads that fail that conditional while all the threads that pass it perform all the instructions in that block of code. Then, when the conditional ends, all the threads begin again performing the remaining instructions in the kernel. Therefore, with EBGTG, if a single thread in a warp is the 'early bird', all other threads will be blocked until that single thread does the slow global memory writes.

We were worried at first that warp divergence would prevent any speedup from EBGTG versus Early Bird Loses the Gradient, but in empirical testing, we saw a consistent 12-15% speedup for the entire program.

2) *Shared Memory*: Earlier versions of cu2rec utilized shared memory to store $userBias$ values. Shared memory on a GPU is accessible by a single block. Thus, every thread in a block can access the same shared memory. When we tested usage of shared memory, we used prefetching to load $userBias$ values from global memory into shared memory. This is done at the start of the kernel. We `__syncthreads` after the loading so no threads in the block can start loading values from shared memory until all values are loaded in. However, we saw a slowdown with prefetching. At first, we thought this was due to not having enough threads in a block to make use of prefetched memory. However, adding more threads per block created more of a slowdown than the prefetching of $userBias$ values did.

E. Advanced Techniques

To optimize run-time and ensure best in class results on test sets, cu2rec utilizes multiple advanced techniques. The use of constant memory, registers, and an optimized reduction sum kernel use the GPU's architecture to greater advantage. Our parallel lock-free updates and user index striding ensure we achieve the best results possible against test set ratings.

1) *Constant Memory*: All of our kernels rely on many of the same static variables, such as hyper-parameters and dimensions. These values include total iterations, number of factors, η , λ_p , λ_q , λ_u , and λ_i .

The constant memory is useful because from the kernel's point of view, it never changes, and can therefore be aggressively cached. Because all of the values we store in constant memory never change during the execution of cu2rec, all of these values get cached in the beginning of execution and remain in the cache for the runtime of the program.

2) *Register Utilization*: Registers are the fastest memory available on the GPU and accesses to this memory can be over 100x faster than global memory[4]. As such, one of our goals was to use registers to store any values that are used more than once in a kernel. We will break down which values we decided to store in registers.

In the SGD Kernel (algorithm 1), the following variables are created to store values in registers: x (the user id), low (the first rated item id index pointer), $high$ (the last rated item id index pointer), yi (random item index pointer), y (the item id), ub (user x 's bias), ib (item y 's bias), $error$ (the error of the model on the rating of user x on item y), $earlyBird$ (if the user is first to update item's features), $pOld$ (old value for feature matrix P at row x and column i), and $qOld$ (old value for feature matrix Q at row y and column i).

In the Loss Kernel (algorithm 2), the following variables are created to store values in registers: x (the user id), ub (user x 's bias), and $itemId$ (the item index for every item x has rated).

We experimented with having less register use, thinking it might allow for more blocks to be scheduled at the same time, but that resulted in more global memory accesses (and more pressure on the caches), and was overall empirically slower.

3) *Reduction Sum*: The total loss kernel, which is used to calculate the global loss after a set number of updates, uses a fixed number of threads t_g per grid regardless of the number of ratings. This makes it easy to scale to very high number of ratings (N). It uses the reduction sum technique, wherein each thread at each step calculates a partial sum of its previous partial sum and the next thread's previous partial sum. This is done for $\log_2(N)$ steps.

Each thread initially calculates the sum of N/t elements sequentially, where each element is t_g apart from the previous one, allowing coalesced memory accesses. These sums are written into the shared memory.

After this step, each block (size t_b) reduces its own sum into the sum at $tid = 0$. This is done for $\log_2(N)$ steps, wherein in the first step, the first $t_b/2$ threads calculate their partial sum with the sums in tid and $tid + t_b/2$, and then $t_b/4$ threads, and so on. This approach coalesces the memory accesses and minimizes branch divergence. We also use loop unrolling to reduce the number of if statements, and also use templating with the block size to get rid of the unnecessary unrolled checks at compile-time.

At the end, each thread in the thread block with $tid = 0$ writes its own sum to the global memory, so we get a partial sum per block. These can be fed into another reduce sum kernel, but we found doing the final addition on the host was faster.

4) *Parallel Lock-Free Updates*: As discussed in the section explaining Early Bird Gets the Gradient, cu2rec does not lock any memory address writes while updating feature matrix weights. Earlier versions of cu2rec utilized atomic operations to update the Q and $itemBias$ matrices. However, after discovering that optimal results could be achieved by simply letting a single thread 'win' an update, we decided to remove any atomic operations from any kernel.

5) *User Index Striding*: EBG TG favors whichever thread selects an item first and sets the $itemIdUpdated$ value to *true*. Therefore, we must ensure that every thread (and therefore every user) gets a fair shot to be the 'early bird.' At first, with naive EBG TG, we were seeing 2-3% worse results on error metrics on test data than other implementations. This gap in results increased for larger datasets, including almost 8% worse results on the Netflix Dataset when compared to best in class results ([13]). We learned that EBG TG was scheduling lower index users first since they would always be in the first few blocks every iteration. While there is no guarantee which blocks begin running first, there are limited SMs on a GPU and therefore some blocks are queued while waiting to run. We found that later users would consistently be scheduled to higher block indexes, and thus run last.

In order to combat this, we decided to offset which user each thread uses to perform SGD. In the training loop, we add an offset variable every iteration and pass that to the SGD kernel (algorithm 1). This effect can be seen in the first line where x is computed. What this offset does is ensure that over time, every thread will be responsible for different sections of the user matrix.

By implementing this striding, we immediately began to see equivalent results in test set error metrics compared to other well known implementations. As far as we know, there are no other GPU implementations that make use of user index striding.

IV. EXPERIMENTAL SETUP

To test the performance and correctness of cu2rec, we compared our implementation against two sequential SGD MF implementations.

The first implementation was written by the authors. It is a naive algorithm that removes the parallelism over each user in the ratings matrix and instead loops over every user for each iteration. For each user, a random item that they have rated is selected. All other implementation decisions were left as close to cu2rec as possible in order to get a fair comparison. Of course, EBG TG was removed as the race condition on Q and $itemBias$ matrices does not exist in a sequential version of SGD.

The second sequential version of SGD MF was the Surprise[9] library, and specifically, its implementation of the biased-SVD MF algorithm[1]. This algorithm also uses SGD to learn the same feature matrices as cu2rec. However, this implementation uses epochs instead of iterations, where an epoch is one pass over all ratings. cu2rec treats an iteration as one pass over all users, picking one item at a time. Thus, we decided to compare cu2rec with Surprise based on the number of updates, and thus floating point operations, to the feature matrices. For cu2rec, each iteration does $|U|$ updates, where U is the set of all users in the dataset. Surprise does $|R|$ updates per epoch, where R is the set of all ratings in the dataset. Surprise takes in number of epochs to train with while cu2rec takes in number of iterations. Therefore, the following equation gives us how many epochs to train Surprise with for fair comparisons:

$$|U| * iterations = |R| * epochs \quad (10)$$

All 3 implementations were tested on three datasets, MovieLens-100k, MovieLens-20M, and Goodreads-105M[17]. The MovieLens datasets have been a standard for comparison of recommender systems for a while, but they are still quite limited in scale. We therefore also opted to use the Goodreads-105M dataset introduced by Wan and McAuley [17], which features around 105 million ratings by 808,749 users on 1,561,465 items. We also opted not to run our sequential implementation on Goodreads-105M due to time constraints.

Each implementation was trained using 1000, 5000, and 10000 iterations (for MovieLens-100k and Goodreads-105M) and 100, 500, 1000, and 5000 iterations (for MovieLens-20M). For each implementation and each experiment of i iterations, we use 50 and 300 factors.

For each experiment, we capture the total time taken for the program to run and the final Root Mean Squared Error (RMSE) on the test set.

$$RMSE = \sqrt{\frac{1}{n} \sum_{i=1}^n (R_i - \hat{R}_i)^2} \quad (11)$$

Each implementation is trained on 80% of the full dataset and 20% is held out for a test set. No ratings from the test set are ever using in any feature matrix weight updates. Each dataset is shuffled using the Python scripts discussed in the Implementation section of this paper. Each implementation is given the same train and test set so that no randomness in the split affects final RMSE.

We tested each implementation with with same hyper-parameters.

$$\begin{aligned} \eta &= 0.01 \\ \lambda_p &= 0.02 \\ \lambda_q &= 0.02 \\ \lambda_u &= 0.02 \\ \lambda_i &= 0.02 \end{aligned}$$

V. RESULTS

We benchmarked our GPU code using an NVIDIA V100 GPU, and the CPU code with an Intel i7-6650U CPU.

Tables I, II, and III show the times and RMSE on MovieLens-100k, MovieLens-20M, and Goodreads-105M dataset respectively.

Figures 2, 4, and 6 show the final test RMSE on MovieLens-100k, MovieLens-20M, and Goodreads-105M dataset respectively.

Figures 3, 5, and 7 shows the graph of the total times of all implementations on MovieLens-100k, MovieLens-20M, and Goodreads-105M dataset respectively.

TABLE I
MOVIELENS 100K - TOTAL TIME AND TEST RMSE SGD MF IMPLEMENTATIONS

ALGORITHM	ITERS	F	TEST RMSE	TOTAL TIME
CU2REC (GPU)	1000	50	.9077	0.65s
		300	.9076	1.78s
	5000	50	.8946	0.75s
		300	.8873	2.30s
	10000	50	.8945	0.87s
		300	.8788	2.95s
CU2REC (CPU)	1000	50	.9061	11.2s
		300	.8907	14.3s
	5000	50	.9041	58.2s
		300	.8747	69.7s
	10000	50	.9054	114.0s
		300	.8695	143.2s
SURPRISE	1000	50	.9078	1.4s
		300	.8724	5.38s
	5000	50	.9305	5.98s
		300	.8545	24.4s
	10000	50	.9237	11.2s
		300	.8608	48.8s



Fig. 2. Comparison of RMSE on ML-100k

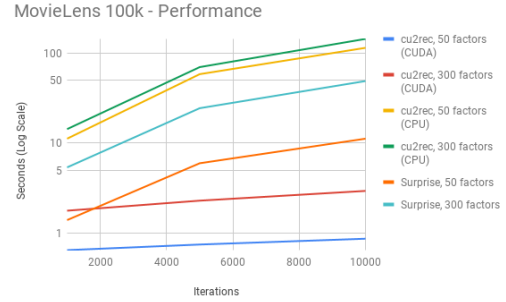


Fig. 3. Comparison of total time on ML-100k

TABLE II
MOVIELENS 20M - TOTAL TIME AND TEST RMSE SGD MF IMPLEMENTATIONS

ALGORITHM	ITERS	F	TEST RMSE	TOTAL TIME
CU2REC (GPU)	100	50	.9004	11.1s
		300	.8999	29.2s
	500	50	.8678	11.5s
		300	.8679	37.7s
	1000	50	.8663	12.6s
		300	.8664	48.6s
CU2REC (CPU)	5000	50	.8417	21.6s
		300	.8044	137.0s
	100	50	.8642	265.6s
		300	.8583	350.1s
	500	50	.8738	1282.7s
		300	NA ¹	NA ¹
SURPRISE	1000	50	NA ¹	NA ¹
		300	NA ¹	NA ¹
	100	50	.8613	329.1s
		300	.8743	721.8s
	500	50	.8791	487.6s
		300	.8342	1075.8s
	1000	50	.8023	694.6s
		300	.8004	1474.8s

INDICATES THAT PROGRAM WAS INFEASIBLE TO RUN¹

A. GPU vs GPU comparison

We tested our code both with a V100 on NYU's Prince server, and a TITAN Z in NYU's cuda2 server. We varied problem sizes, the memory bandwidth requirements, and the

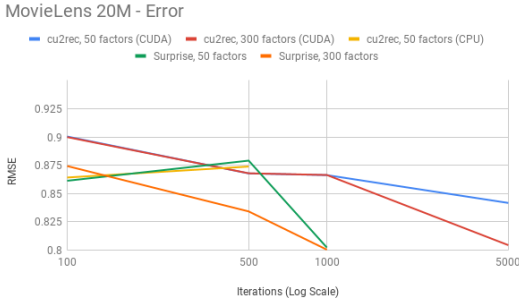


Fig. 4. Comparison of RMSE on ML-20M

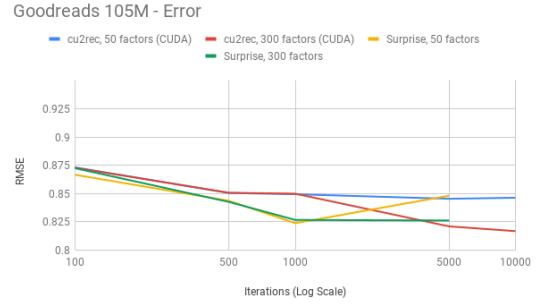


Fig. 6. Comparison of RMSE on Goodreads-105M

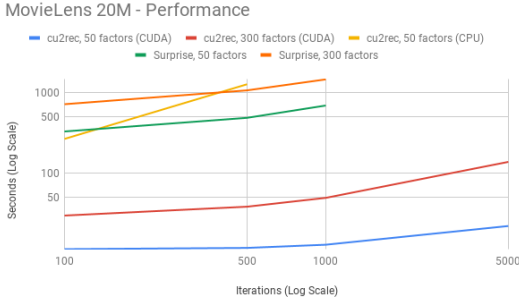


Fig. 5. Comparison of total time on ML-20M

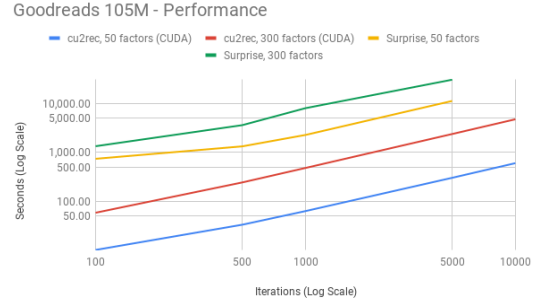


Fig. 7. Comparison of total time on Goodreads-105M

TABLE III
GOODREADS 105M - TOTAL TIME AND TEST RMSE SGD MF IMPLEMENTATIONS

ALGORITHM	ITERS	F	TEST RMSE	TOTAL TIME
CU2REC (GPU)	100	50	.8731	10.30s
		300	.8728	59.17s
	500	50	.8505	33.70s
		300	.8507	245.86s
	1000	50	.8494	63.94s
		300	.8499	485.29s
	5000	50	.8454	305.65s
		300	.8210	2,393.39s
	10000	50	.8463	608.15s
		300	.8168	4,782.57s
SURPRISE	100	50	.8666	746.50s
		300	.8724	1,348.50s
	500	50	.8436	1,339.70s
		300	.8426	3,630.30s
	1000	50	.8238	2,289.50s
		300	.8267	8,063.30s
	5000	50	.8481	11,321.40s
		300	.8262	30,733.10s
	10000	50	NA ¹	NA ¹
		300	NA ¹	NA ¹

INDICATES THAT PROGRAM WAS INFEASIBLE TO RUN¹

number of iterations to get a healthy comparison. This is visualized in figure 8.

The V100 has 5,120 CUDA cores vs 5,760 of TITAN Z, 900GB/s memory bandwidth vs 672GB/s of TITAN Z, and

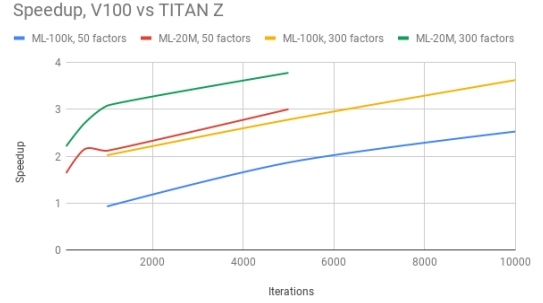


Fig. 8. Comparison of speedup on different problem sizes

higher clock speed.

In terms of the cost in performance, going from the ML-100k to ML-20M dataset increases the total problem size, as there are more users, items, and ratings to train for. Holding all other things equal, we would expect a scalable code to have higher speedup with larger problem size, and our speedup satisfies this.

In terms of bandwidth requirement, increasing the number of factors from 50 to 300 has a direct effect on it, because the SGD kernel needs to retrieve more data from the global memory. Holding all other things equal, we would expect a scalable code to have higher speedup with increased memory bandwidth, and our speedup satisfies this.

Note that we do not utilize the tensor cores found in the V100 (in fact, we compile for compute capability 3.5).

B. CuMF-ALS

CuMF-ALS[14] does not give a general purpose script to convert data into a format that is readable by their program, so we had to use the data that they provide in order to test their code. The link to download the Netflix data[3] doesn't work, but we were able to get it through their other repo. Using the command line options and hyper-parameters in their readme, we were able to run the code and get similar error metrics (0.91 RMSE) to the paper. However, the command line options they give mixes up the number of users and the number of items, and the program failed to run when this error was corrected. Therefore, we could not verify the accuracy of their claims.

We tried the MovieLens[8] datasets, specifically the MovieLens-10M dataset as it is one of their preferred datasets. However, MovieLens-10M produced *NaN* results, and when the option to suppress *NaN*s was used, the training set *RMSE* started with 1.56 in the first iteration, increased to 3.58, and stayed there.

C. CuMF-SGD

CuMF-SGD[13] was easier to work with, and accepted our training datasets. However, it did not output any error metrics, meaning we had to rely on the metrics in the paper, which produces a similar result to CuMF-ALS on Netflix data. The paper does not provide any error metrics on MovieLens datasets, which is the primary dataset we use to benchmark our results.

D. Implicit

Implicit[15], as mentioned in related work, uses implicit ratings and uses accuracy (whether a user has consumed an item or not) as its error metric. Attempting to convert all explicit ratings to implicit ratings of 1 (consumed) creates an interesting dilemma wherein all of the training and test ratings are 1, and therefore, a model that only predicts 1 would have a perfect (0) RMSE. When we tried this approach, we were indeed able to get a very low RMSE.

The only other option is to have a cutoff point for consumption, wherein high ratings would get 1 and low ratings would get 0, and we would still keep these 0 ratings in our training and test sets. That, however, goes against the spirit of implicit ratings.

Because we explicitly optimize for RMSE, it doesn't make sense for us to calculate implicit error metrics. Therefore, we couldn't find a valid way of comparing our work.

VI. CONCLUSION

GPU technology opens the door to massive acceleration for a wide variety of problems. Machine Learning has recently seen a massive boost in effectiveness from both powerful GPUs and the availability of big data to train complex models on. Recommender Systems are an interesting subset of machine learning as they benefit greatly from larger and larger datasets that allow models to uncover complex latent relationships between users and items. SGD is one of the most

popular algorithms to optimize recommender system MF models. However, SGD provides a challenging problem for GPU implementations due to its inherent sequential definition[14].

This paper introduced cu2rec, a novel parallel implementation of SGD used to solve recommender system matrix factorization. Through the use of a parallel lock free SGD kernel and a variety of advanced CUDA programming techniques, cu2rec achieves a 10x speedup over one of the fastest sequential recommender system MF libraries while matching best in class error metrics. In addition to outperforming sequential implementations of SGD MF, cu2rec has also been shown to scale with better GPU hardware.

A. Future Work

In the future, we plan to improve the parallelization capability of our program by allowing multi-GPU support. Similarly, we plan to add support for extra large datasets through the use of tiling, with overlapping communication and execution through the use of streams. We also plan to make more use of CUDA streams, such as overlapping the calculation of the two total loss metrics with execution of SGD.

The use of half precision floating points or unsigned integers instead of floating points has been known to speed up execution massively[13]. That is one area that we expect to give us a major increase in speed.

Lastly, we saw that GPU implementations of MF[14][15] parallelize over the factors instead of users. That parallelization requires more advanced kernels and would require a rework of the EBGTE (1) algorithm. However, it is an avenue we want to keep exploring.

1) *Algorithms*: Alternating Least Squares[16] is a different way of training a MF model that does not result in race conditions even in a parallel implementation because it only updates one matrix at a time. It is therefore easy to parallelize[14], but SGD has higher potential for finer-grained parallelism[11].

We would also like to try out implicit ratings, as they are the de-facto industry standard for recommender systems. That is because users rarely interact with items that they know they are not going to like, bringing the average ratings up so high that their average ratings center around 4/5. That would require us to use a different loss function and different error metrics.

We also want to explore different gradient descent optimizers, such as Adam[18] or the newer AdamW[19]. The downside of these optimizers is keeping track of the momentum terms for each trainable parameter, doubling the memory necessary to hold the matrices.

REFERENCES

- [1] Y. Koren, R. Bell, and C. Volinsky, "Matrix factorization techniques for recommender systems," *Computer*, no. 8, pp. 30–37, 2009.
- [2] A. Bari, M. Chaouchi, and T. Jung, *Predictive analytics for dummies*. John Wiley & Sons, 2016.
- [3] Y. Koren, "The bellkor solution to the netflix grand prize," *Netflix prize documentation*, vol. 81, pp. 1–10, 2009.
- [4] D. B. Kirk and W. H. Wen-Mei, *Programming massively parallel processors: a hands-on approach*. Morgan kaufmann, 2016.

- [5] A. Coates, B. Huval, T. Wang, D. Wu, B. Catanzaro, and N. Andrew, "Deep learning with cots hpc systems," in *International Conference on Machine Learning*, 2013, pp. 1337–1345.
- [6] S. Funk, "Netflix update: Try this at home," December 2006, [Online; posted December 11, 2006]. [Online]. Available: <http://sifter.org/simon/journal/20061211.html>
- [7] G. H. Golub and C. Reinsch, "Singular value decomposition and least squares solutions," *Numerische mathematik*, vol. 14, no. 5, pp. 403–420, 1970.
- [8] F. M. Harper and J. A. Konstan, "The movielens datasets: History and context," *ACM Trans. Interact. Intell. Syst.*, vol. 5, no. 4, pp. 19:1–19:19, Dec. 2015.
- [9] N. Hug, "Surprise, a Python library for recommender systems," <http://surpriselib.com>, 2017.
- [10] B. Recht, C. Re, S. Wright, and F. Niu, "Hogwild: A lock-free approach to parallelizing stochastic gradient descent," in *Advances in neural information processing systems*, 2011, pp. 693–701.
- [11] H. Yun, H. Yu, C. Hsieh, S. V. N. Vishwanathan, and I. S. Dhillon, "NOMAD: non-locking, stochastic multi-machine algorithm for asynchronous and decentralized matrix completion," *CoRR*, vol. abs/1312.0193, 2013. [Online]. Available: <http://arxiv.org/abs/1312.0193>
- [12] Y. Low, D. Bickson, J. Gonzalez, C. Guestrin, A. Kyrola, and J. M. Hellerstein, "Distributed graphlab: a framework for machine learning and data mining in the cloud," *Proceedings of the VLDB Endowment*, vol. 5, no. 8, pp. 716–727, 2012.
- [13] X. Xie, W. Tan, L. L. Fong, and Y. Liang, "Cumf_sgd: Fast and scalable matrix factorization," *CoRR*, vol. abs/1610.05838, 2016. [Online]. Available: <http://arxiv.org/abs/1610.05838>
- [14] W. Tan, L. Cao, and L. L. Fong, "Faster and cheaper: Parallelizing large-scale matrix factorization on gpus," *CoRR*, vol. abs/1603.03820, 2016. [Online]. Available: <http://arxiv.org/abs/1603.03820>
- [15] B. Frederickson, "Fast python collaborative filtering for implicit feedback dataset," <https://github.com/benfred/implicit>, 2016.
- [16] G. Takács, I. Pilászy, and D. Tikk, "Applications of the conjugate gradient method for implicit feedback collaborative filtering," in *RecSys*, 2011.
- [17] M. Wan and J. McAuley, "Item recommendation on monotonic behavior chains," in *Proceedings of the 12th ACM Conference on Recommender Systems*. ACM, 2018, pp. 86–94.
- [18] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *arXiv preprint arXiv:1412.6980*, 2014.
- [19] I. Loshchilov and F. Hutter, "Fixing weight decay regularization in adam," *arXiv preprint arXiv:1711.05101*, 2017.